

→ Model Tuning! OR (Hyperparameters) Tuning.
→ Ensemble Learning! Like using multiple models (combine) to see the best model, or which model we have to choose for prediction.
Model Tuning and Ensemble Learning Technically both work together.

=> Model Tuning! Means fine tuning like we have some ML Model to whom we make fine tune.

But in ML:

Model something that predict so we have to tune that model more,

til now, we have learned how to build machine learning models - but there is one big truth about models:

The first version of your model is never the best version.

That's where Model Tuning comes in.

Tuning
like

Hyperparameters: (n neighbors = 5, weight = 'uniform', leaf size = 30, $p=2$)

also by changing hyperparameter and then we see which parameter given the best result, or best accuracy score for all model then we use that

But if we just use random hyperparameter our model might underfit, or overfit, or simply perform poorly

also the goal of Model Tuning is.

1. To find the best combination of hyperparameters.

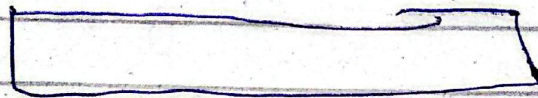
2. so that our model performs better on new, unseen data - not just on the training data.

Before Hyperparameter Tuning.
we have to learn Cross Validation

⇒ Cross Validation?

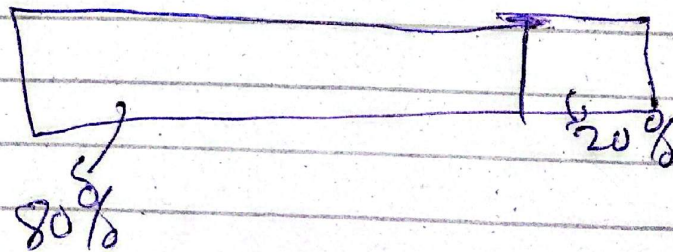
Step 1 let's learn train test split

we have our data.



100% of my data.

so if i want train test split
we create one line 20%



not compulsory. we always split
it into the 80% 20%.

so 80% is known as our
training data.

and 20% is known as our testing
data.

So we trained OUR Model
with 80% data.

and then we Test (Predict)

and On the basis On that we create model

So when we are talking about Cross Validation

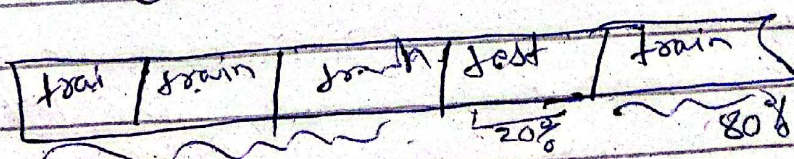
it is possible that 20% data you split might be good for training data.

So if that happen, then our model should be, Overfit, Underfit, poor fit.

So there are multiple factors so we can't rely only on 80% and 20%

So in cross validation we take 100% of data, and here we are using K-fold cross validation.

So in K-fold cross validation we divide our 100% data as K parts. like $K=5$



so this time this will become as 80% and 20%

so we will do training and testing in different different ways.

and this is the core concept of cross validation.

Means we continuously swap data.

Step 1: portion one for test then portion 2

so in cross-v we divide Data in K parts and every part will be your testing data, remaining will be your training data.

then we check score on each iteration.

Where Accuracy score goes high then we will select high.

Note: While implementing cross validation we get multiple Accuracy score and the get Average (combination) of those scores.

so we will choose Average Accuracy score.

so this is how cross validation works.

→ Note: in practical implementation we don't need to split again and again, because in Python there is a cross validation library you just have to use.

step 1

df

⇒ Data



Model



// can be overfit/underfit. so we use

cross validation



Grid Search CV

↳ then we create perfect model.